

CSEN604: Data Bases II

Mohamed Karam Gabr

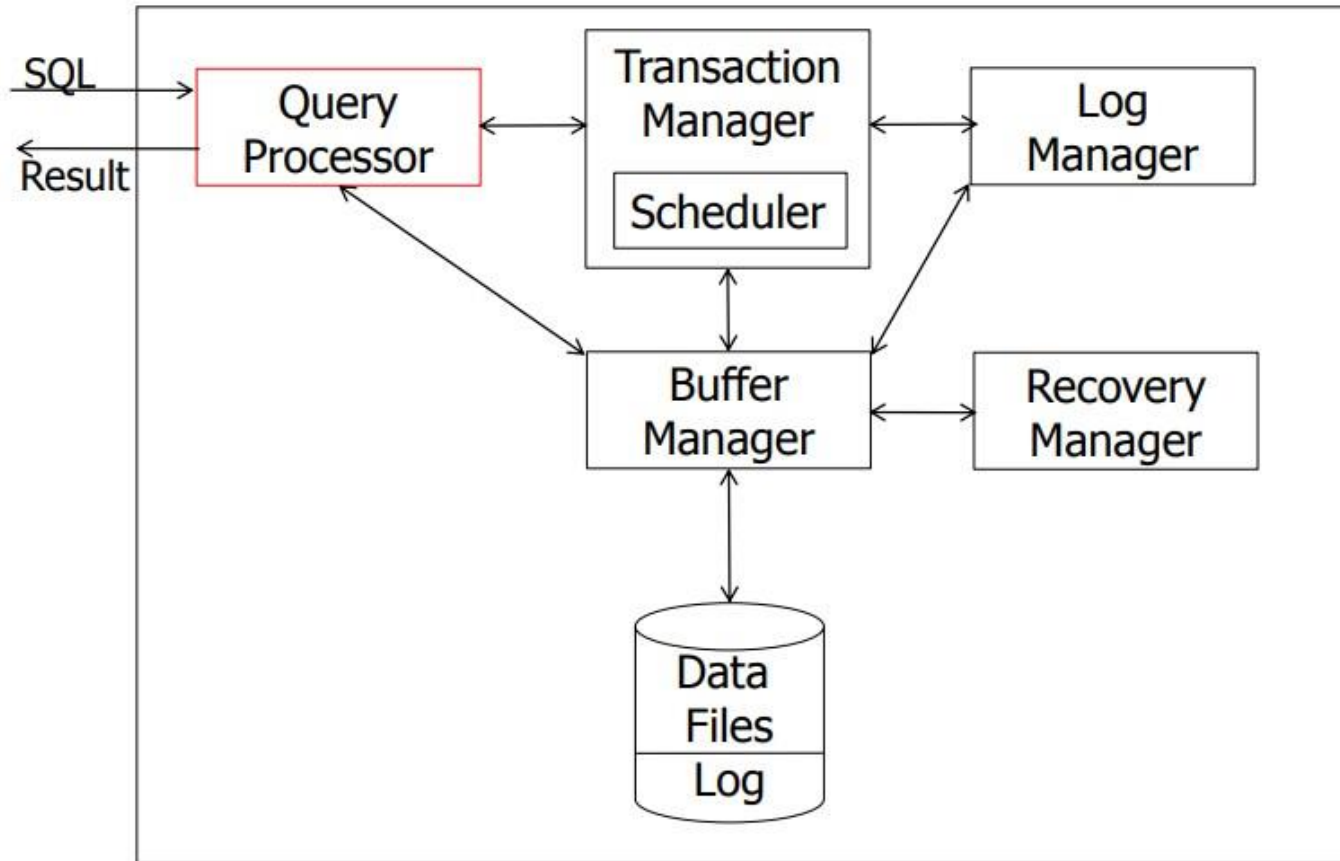
Physical Plan

Spring 2025

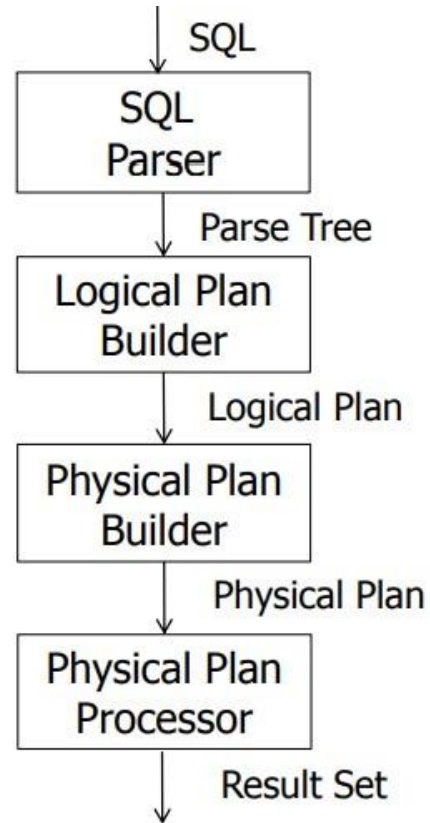
Outline

- Physical plan.
- Building Physical plans.
- Iterators.
- Operators.
- Relation Access Algorithms.

Introduction

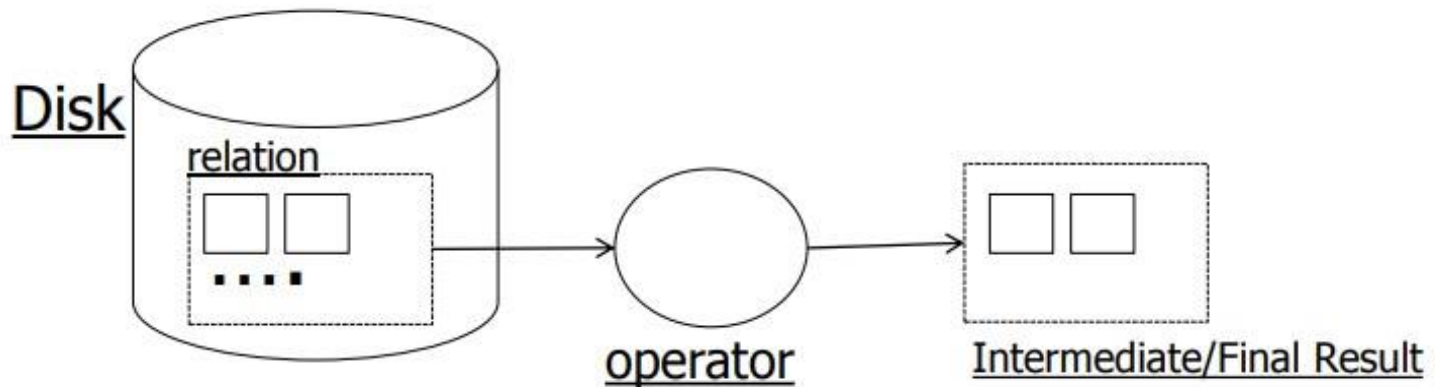


Introduction



Physical plan

- To build a physical plan, you will need:
Relations.
+
Operators which act on relations producing intermediate or final results.



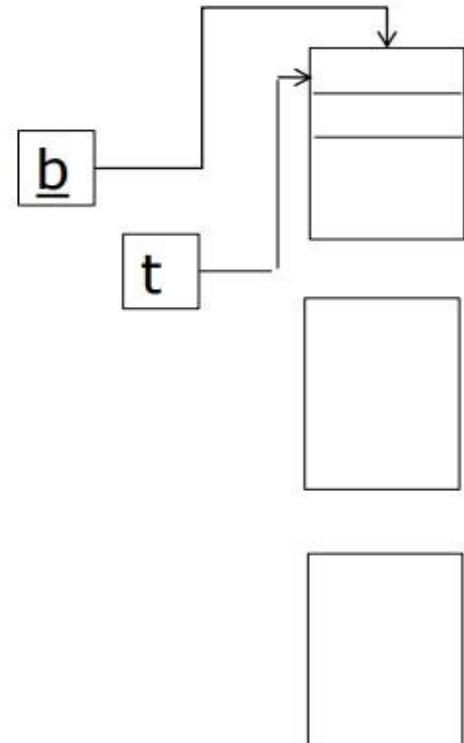
- Physical plan operators can be:
 - Basic set of operators that define the language of physical query execution plans.
 - Comprises the set of relational operators + some more required operators to be actually able to execute a query.
 - Many physical operators can be implemented as an iterator.

Iterators

- It is a group of 3 methods that allows a consumer of the result of the physical operator to get the result one tuple at a time.
- When describing iterators and their methods, we shall assume that there is a “class” for each type of iterator (i.e., for each type of physical operator implemented as an iterator), and the class defines Open(), GetNext(), and Close() methods on instances of the class.
- Many physical operators can be implemented as an iterator.
- Iterators are the basic building blocks of a physical plan. Table scan (also called SEQ scan) and index scan are the most common ones.
- The simplest iterator is the one that implements the table-scan operator.

Table-scan Iterator

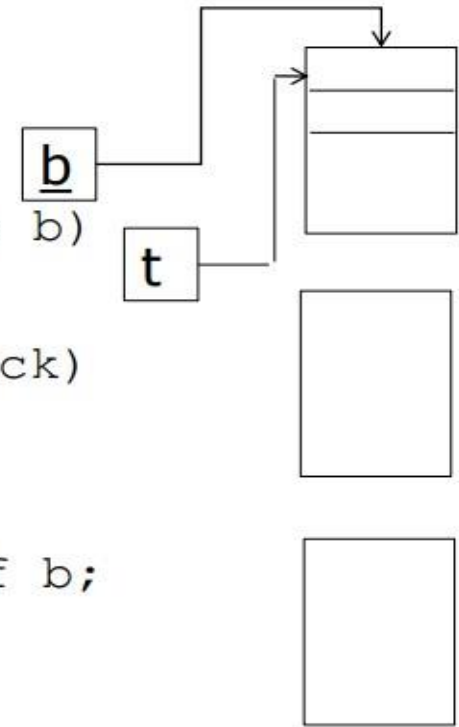
```
Open() {  
    b <- first block of R;  
    t <- first tuple of b;  
}  
  
close() {  
}
```



- `Open()`: starts the process of getting tuples but does not get a tuple. It initializes any data structures needed to perform the operation.
- `Close()`: ends the iteration after all tuples, or all tuples that the consumer wanted, have been obtained

Table-scan Iterator

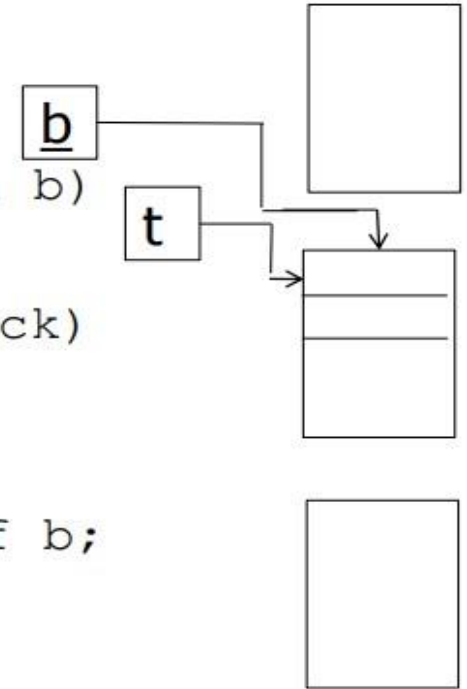
```
getNext() {  
    if( t is beyond the last tuple in b)  
        increment b;  
    if (b is beyond last block)  
        return NoMoreData;  
    else  
        t <- first tuple of b;  
  
    oldt <- t;  
    increment t;  
    return oldt;  
}
```



- `getNext()`: returns the next tuple in the result and adjusts data structures to allow subsequent tuples to be obtained. If there are no more tuples to return.
- `getNext()`: If there are no more tuples to return, it returns a special value `NotFound`, which we assume cannot be mistaken for a tuple.

Table-scan Iterator

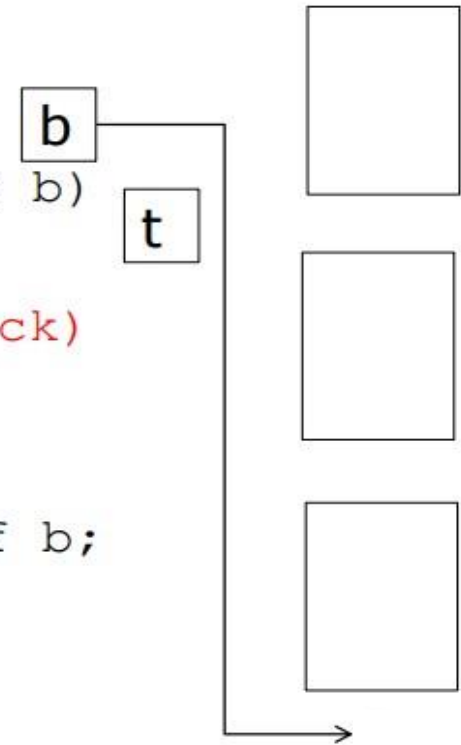
```
getNext() {  
    if( t is beyond the last tuple in b)  
        increment b;  
    if (b is beyond last block)  
        return NoMoreData;  
    else  
        t <- first tuple of b;  
  
    oldt <- t;  
    increment t;  
    return oldt;  
}
```



- `getNext()`: returns the next tuple in the result and adjusts data structures to allow subsequent tuples to be obtained. If there are no more tuples to return.
- `getNext()`: If there are no more tuples to return, it returns a special value `NotFound`, which we assume cannot be mistaken for a tuple.

Table-scan Iterator

```
getNext() {  
    if( t is beyond the last tuple in b)  
        increment b;  
    if (b is beyond last block)  
        return NoMoreData;  
    else  
        t <- first tuple of b;  
  
    oldt <- t;  
    increment t;  
    return oldt;  
}
```

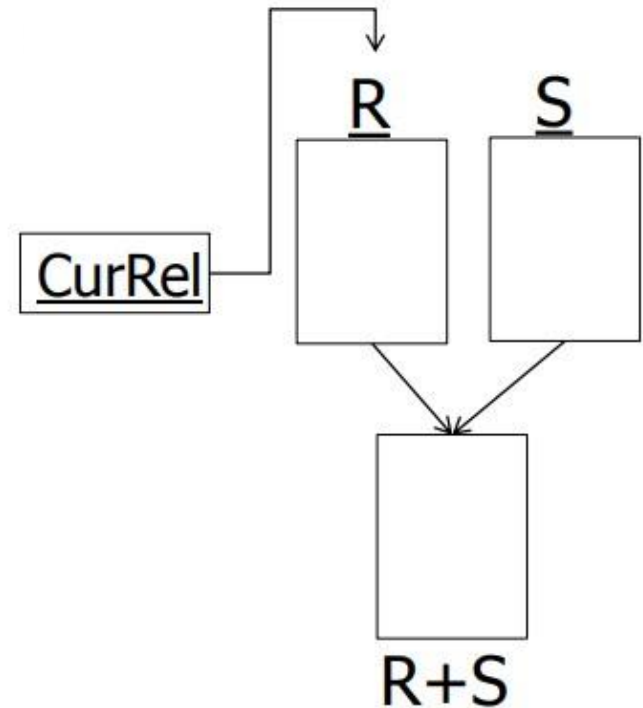


- `getNext()`: returns the next tuple in the result and adjusts data structures to allow subsequent tuples to be obtained. If there are no more tuples to return.
- `getNext()`: If there are no more tuples to return, it returns a special value `NotFound`, which we assume cannot be mistaken for a tuple.

Bag Union Iterator (R+S)

The methods use a shared variable `CurRel` that is either `R` or `S`, depending on which relation is being read from currently.

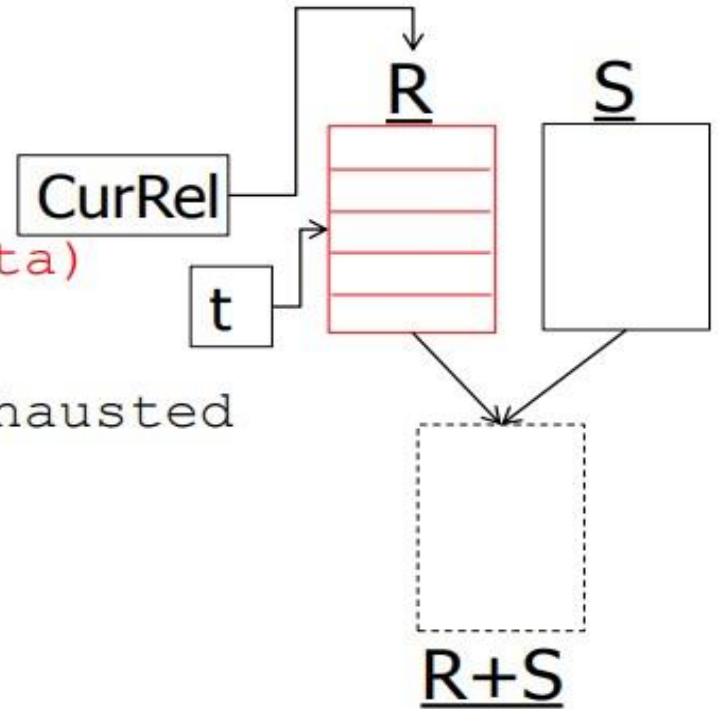
```
Open () {  
    R.open () ;  
    CurRel <- R ;  
}  
  
close () {  
    R.Close ( ) ;  
    S.Close ( ) ;  
}
```



Bag Union Iterator (R+S)

The methods use a shared variable `CurRel` that is either `R` or `S`, depending on which relation is being read from currently.

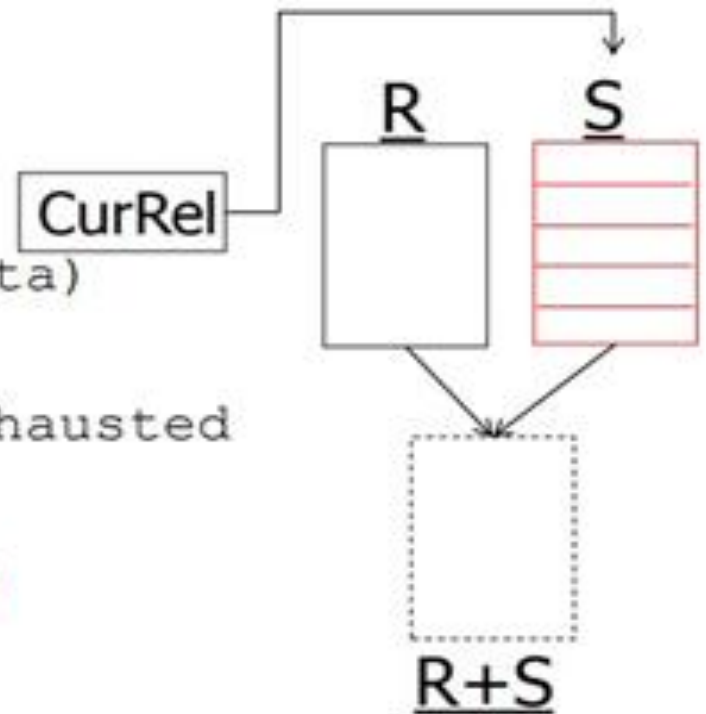
```
getNext() {  
    if (CurRel == R) {  
        t <- R.getNext();  
        if (t != NoMoreData)  
            return t;  
        else { // R is exhausted  
            S.Open();  
            CurRel <- S;  
        }  
    }  
    else return S.getNext();  
}
```



Bag Union Iterator (R+S)

The methods use a shared variable `CurRel` that is either `R` or `S`, depending on which relation is being read from currently.

```
getNext() {  
    if (CurRel = R) {  
        t <- R.getNext();  
        if ( t != NoMoreData)  
            return t;  
        else { // R is exhausted  
            S.Open();  
            CurRel <- S;  
        }  
    }  
    else return S.getNext();  
}
```



Using Cursors from High level languages

All relational databases would expose their internal cursors to enable gradual (one row at a time) access to the result set

```
String query = "select COF_NAME, SUP_ID, PRICE, SALES, TOTAL from  
                dbName.COFFEES";  
  
try {  
    stmt = con.createStatement();  
    ResultSet rs = stmt.executeQuery(query);  
    while (rs.next()) {  
        String coffeeName = rs.getString("COF_NAME");  
        int supplierID    = rs.getInt("SUP_ID");  
        float price       = rs.getFloat("PRICE");  
        int sales         = rs.getInt("SALES");  
        int total         = rs.getInt("TOTAL");  
        System.out.println(coffeeName + "\t" + supplierID + "\t" +  
                            price + "\t" + sales + "\t" + total);  
    }  
}
```

Java

Classifying Operators

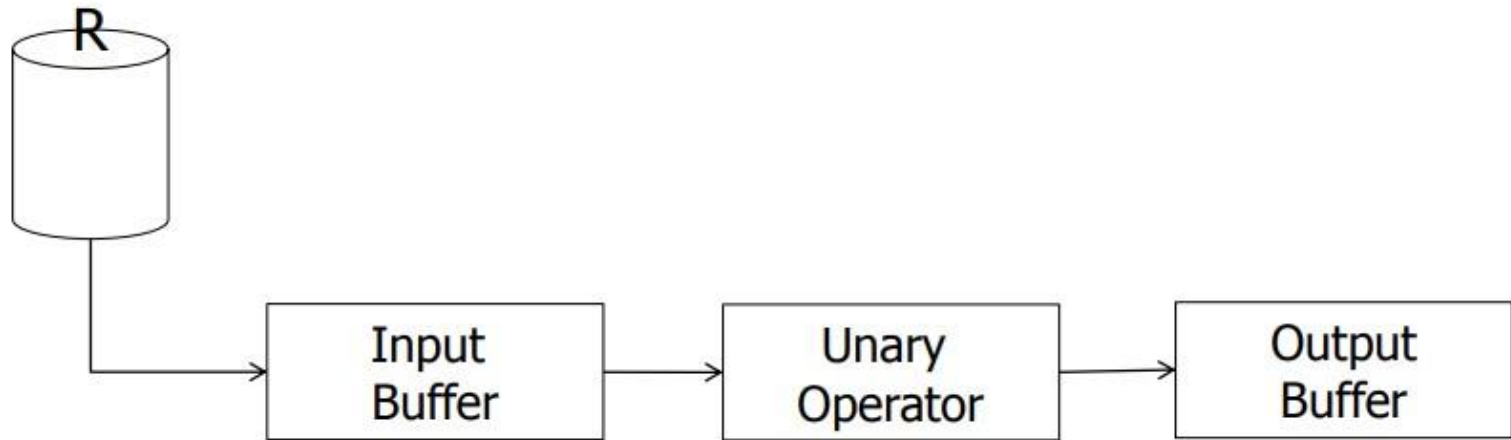
- By input type:
 - **Tuple-at-a-time operators:** Can read only one block/page at a time and required to work only with one tuple. e.g., projection.
 - **Full-relation operators:** required knowledge of all or most of the relation. e.g., distinct.
 - **Full-relation binary operators:** same as above, but on two relations. e.g., intersection.

Classifying Operators

- By algorithm type:
 - **One Pass Operator:**
 - When one of the relations fits completely into memory.
 - In addition, there is enough memory to store at least one block/page of the other relation.
 - If M memory buffers are available, then one of the relations should have a maximum size of $M-1$.
 - If too few buffers are allocated, then one-pass algorithms may not run.
 - Relies on correctly estimating relation sizes and allocating memory buffers
 - **Multi-pass Operator:**
 - When entire relations cannot be read into memory.
 - Requires alternate computation and retrieval of intermediate results.
 - Many multi-pass algorithms are generalizations of their corresponding two-pass algorithms.

Tuple-at-a-time Unary Operators

- Example: $\sigma_{(R)}$, $\Pi_{(R)}$
 - A strategy for a one-pass algorithm



Relation-at-a-time Binary Operators

- One-pass strategies for binary relation at-a-time operators vary between different operators.
- Almost all of them require at least one of the relations to be completely stored in memory.
- Operators:
 - $R \cup S$
 - $R - S$
 - $S - R$
 - $S \times R$
 - $S \bowtie R$
 - $R \wedge S$

Relation-at-a-time Binary Operators

- Set Union $R \cup S$
- Assuming R is the **bigger** relation:
 - Read S into memory completely and make it accessible through an in-memory index structure
 - output all tuples of S while reading
 - For each tuple f in R , search if it already exists in S , and output if not.
- Set Difference $R - S$
- Assuming R is the bigger relation:
 - Read S into memory completely and make it accessible through an in-memory index structure
 - For each tuple of R , search if it already exists in S
 - If tuple exists in S , then ignore, else output

Relation-at-a-time Binary Operators

- Set Difference $S - R$
- Assuming R is the bigger relation:
 - Read S into memory completely and make it accessible through an in-memory index structure
 - For each tuple f in R , search if it already exists in S , and delete it from S if it exists
 - output all remaining tuples of S
- Cross Product $R \times S$
- Assuming R is the bigger relation:
 - Read S into memory completely and store it in a buffer.
 - No special data structure is required.
 - For each tuple f in R , combine it with each tuple of S and output the result.

Relation-at-a-time Binary Operators

- Set Intersection $R \wedge S$
- Assuming R is the bigger relation:
 - Read S into memory completely and make it accessible through an in-memory index structure
 - For each tuple f in R , search if it already exists in S , and output if true.
- Natural Join $R \bowtie S$
- Assume $R(X, Y)$ and $S(Y, Z)$
- Assuming R is the bigger relation:
 - Read S into memory completely and store it in a balanced tree index or a hashtable.
 - For each tuple of R , search S to see if a matching tuple exists
 - Output if matching tuple found.

Relation Access Algorithms

- How should we execute each of the individual steps of a logical query plan?
- The choice of algorithm for each operator is an essential part of the process of transforming a logical query plan into a physical query plan.
- For every SQL based functionality there is an implementation with all the following algorithms/methods.
- Algorithms for database access can be broadly divided in the following categories:
 - Linear-access methods.
 - Sorting-based methods.
 - Hash-based methods.
 - Index-based methods.

Linear Access Methods

- Simplest to implement but the most expensive ($O(n)$).
- Scans the relation, page by page, row by row: Implemented using SEQ Scan operator.
- Note there will be one such linear method implemented for each SQL operation.
- E.g. linear access for duplicate elimination, linear access for selection, etc..
- Sort-based Two-pass Algorithm:
 - Suppose relation R is too big to fit in memory which can accommodate only M blocks of data.
 - The sorting-based two-pass algorithm have the following basic structure.
 - Where data from the operand relations is read into main memory, processed in some way, written out to disk again, and then reread from disk to complete the operation.

Sort-based Two-pass Algorithm

1. Read M blocks of records into memory and sort them.
2. Write them back to disk.
3. Continue steps 1 and 2 until R is exhausted.
4. Use a merge technique to extract relevant results from all the sorted M -blocks on disk (second path).

Example for duplicate elimination:

1. Let relation R , in which duplicates have to be eliminated, be too big to fit in memory.
2. Read $M \ll R$ blocks into memory and sort them.
3. Store the sorted set of M blocks back on disk.
4. Take one block from each sorted sub-list on disk and eliminate duplicate occurrences of tuples.

i.e. take the first element p of the first block and move all other blocks to go beyond p .

Sort-based Algorithm for Duplicate Elimination

Suppose that tuples are integer numbers, only two tuples fit on a block, $M = 3$, and R has 17 tuples:

R								
2,5	2,1	2,2	4,5	4,3	4,2	1,5	2,1	3
1,2	2,2	2,5	2,3	4,4	4,5	1,1	2,3	5
R_1			R_2			R_3		

First Pass.

1. Read the first 3 blocks of R (6 tuples) into the 3 main memory buffers;
2. Sort them and write the sorted sublist R_1 back to the disk;
3. In a similar way generate the sorted sublists R_2, R_3 .

Sort-based Algorithm for Duplicate Elimination

Second Pass – Cycle 1.

1. Read in main memory the first block from every sorted sublist:

Sublists	Main Memory	Waiting on Disk	
R_1	1, 2	2, 2	2, 5
R_2	2, 3	4, 4	4, 5
R_3	1, 1	2, 3	5

2. The least tuple of the three blocks in main memory is 1.
3. We copy 1 to the output:

Output Buffer

1				
---	--	--	--	--

Sort-based Algorithm for Duplicate Elimination

Second Pass – Cycle 1 (cont.).

- Remove the tuple 1 from all the blocks in memory.

Sublists	Main Memory	Waiting on Disk	
R_1	2	2, 2	2, 5
R_2	2, 3	4, 4	4, 5
R_3		2, 3	5

- Since the buffer for R_3 is exhausted we read the next block from this sublist in main memory:

Sublists	Main Memory	Waiting on Disk	
R_1	2	2, 2	2, 5
R_2	2, 3	4, 4	4, 5
R_3	2, 3	5	

- Since there are still tuples in memory go back to step 2.

Sort-based Algorithm for Duplicate Elimination

Second Pass – Cycle 2.

2. The least tuple of the three blocks in main memory is now 2.
3. We copy 2 to the output:

Output Buffer

1	2			
---	---	--	--	--

4. Remove the tuple 2 from all the blocks in memory:

Sublists	Main Memory	Waiting on Disk	
R_1		2, 2	2, 5
R_2	3	4, 4	4, 5
R_3	3	5	

Sort-based Algorithm for Duplicate Elimination

Second Pass – Cycle 2 (cont.).

5. Since the buffer for R_1 is exhausted we read the next block from this sublist, and since there are tuples 2 we remove them as well;

Sublists	Main Memory	Waiting on Disk
R_1	5	
R_2	3	4, 4 4, 5
R_3	3	5

6. Since there are still tuples in memory go back to step 2.

To complete the process, the tuples 3, 4, 5 are consumed in succession giving the desired output:

Output Buffer

1	2	3	4	5
---	---	---	---	---

Hash-based Algorithms

1. Read relation R block by block.
 2. Take each tuple in a block and hash it to one of a set of buckets of a hash file.
 3. All similar tuples should hash to the same bucket.
 4. Examines each bucket in isolation to produce the final result.
- **Hash-based duplicate elimination**, eliminate duplicates from relation R that is too big to fit in memory:
 - Read relation R block by block.
 - Take each tuple in a block and hash it to one of a set of buckets of a hash file.
 - Visit each bucket and eliminate duplicates using a two-pass sort-based algorithm (if bucket too big).
 - **The Hash-Join Algorithm:**
 - Natural Join over relations $R(X,Y)$ and $S(Y,Z)$ is achieved by hashing using just the Y component of R and S.
 - All tuples having the same value of the Y component should hash to the same bucket.
 - If same hash function is used for R and S, corresponding buckets can be compared.
 - Use a one-pass join algorithm by using corresponding buckets from the hash files of R and S.

Index-based Algorithms

- Useful when tuples have to be extracted from relations based on attributes that have been indexed.
- Can be used for index-based selection and index-based join.
- The Zig-Zag join:
 - $R(X,Y) * S(Y,Z)$
 - On 2 B+ trees together
 - At best: Read one page from B+ tree₁ followed by one page from B+tree₂
 - Use intermediate level
Values to learn about
boundaries (skip if needed)
 - Output common

